

LAPORAN TUGAS BESAR
SISTEM OPERASI WAKTU NYATA (RTOS)
Simulasi Sistem Kontrol Lift Multi-Lantai Berbasis FreeRTOS
dengan Priority-Based Scheduling dan Fault-Tolerant Emergency
Handling



Kelompok: P2K

1. Sandy Hidayatullah (101032300009) — [Bagian IDE dan Codingan]
2. Andreas Pangjiashun (101032300002) — [Bagian Simulasi dan Laporan]
3. Mohamad Nur Taufiq (101032300058) — [Bagian Simulasi dan Laporan]

Dosen Pengampu: MUHAMMAD FARIS RURIAWAN, S.T., M.T.

UNIVERSITAS TELKOM
FAKULTAS TEKNIK ELEKTRO
PROGRAM STUDI S1 TEKNIK KOMPUTER
SEMESTER GENAP — TAHUN AKADEMIK 2025/2026

ABSTRAK

Proyek ini mengimplementasikan sistem kontrol lift 5 lantai berbasis *Real-Time Operating System* (FreeRTOS) pada mikrokontroler ESP32 menggunakan simulator Wokwi. Fokus utama penelitian ini adalah penerapan *priority-based preemptive scheduling* untuk menangani berbagai *task* sistem secara konkuren dengan deterministik. Sistem dirancang dengan mekanisme *fault-tolerant emergency handling* menggunakan teknik *deferred interrupt processing* untuk memisahkan logika *ISR* yang kritis dengan penanganan *emergency* yang kompleks. Komunikasi antar-*task* dikelola melalui *queue* untuk *floor request*, sementara integritas data *shared state* (posisi lift, status pintu, dan mode darurat) dilindungi menggunakan *mutex* dengan fitur *priority inheritance* untuk mencegah *priority inversion*. Hasil simulasi menunjukkan bahwa sistem mampu melakukan manajemen *task* secara responsif, menjaga stabilitas *stack*, dan menangani kondisi *race condition* dengan efektif.

AI EDITING STATEMENT

Saya menyatakan bahwa saya menggunakan alat kecerdasan buatan (AI) hanya untuk keperluan editing, termasuk perbaikan tata bahasa, penyusunan kalimat, konsistensi istilah, dan peningkatan keterbacaan dokumen ini.

Seluruh ide, analisis, perhitungan, desain, dan konten substantif merupakan hasil pekerjaan saya sendiri. AI tidak digunakan untuk menghasilkan konten inti, menyelesaikan tugas teknis, atau membuat analisis secara otomatis.

Sebagai bentuk transparansi, berikut adalah daftar penggunaan AI beserta nama alat AI yang digunakan:

No.	Bagian yang Diedit	Jenis Bantuan AI	Nama AI
1	Codingan	Memperbaiki algoritma yang salah	Gemini, Claude

No.	Nama Mahasiswa	NIM	TTD
1	Sandy Hidayatullah	101032300009	
2	Andreas Pangjiashun	101032300002	
3	Mohamad Nur Taufiq	101032300058	

BAB I PENDAHULUAN

1.1 Latar Belakang

Sistem kontrol lift merupakan aplikasi embedded system yang membutuhkan tingkat determinisme dan responsivitas tinggi. Kesalahan dalam menangani request pengguna atau kegagalan sistem saat kondisi darurat dapat berakibat fatal. Penggunaan Real-Time Operating System (RTOS) menjadi solusi standar dalam industri untuk mengelola task yang bersifat konkuren dan asinkron. Dengan RTOS, pengembang dapat memastikan task dengan prioritas tertinggi (seperti sistem darurat) akan selalu mendapatkan waktu prosesor saat dibutuhkan, terlepas dari beban kerja sistem lainnya.

1.2 Rumusan Masalah

1. Bagaimana mengimplementasikan priority-based scheduling pada sistem kontrol lift 5 lantai menggunakan FreeRTOS?
2. Bagaimana mekanisme komunikasi antar-task yang aman untuk menghindari race condition pada data shared state?
3. Bagaimana merancang sistem *fault-tolerant* yang mampu merespons kondisi darurat secara *real-time* tanpa mengganggu *thread* lainnya?

1.3 Tujuan

- Umum: Membangun simulasi sistem kontrol lift lima lantai yang andal dan aman dengan mengimplementasikan konsep Real-Time Operating System (FreeRTOS) pada platform mikrokontroler ESP32.
- Khusus :
 - Mengimplementasikan *priority-based preemptive scheduling* untuk menjamin determinisme sistem.
 - Menerapkan mekanisme sinkronisasi data (*mutex* dan *queue*) untuk memastikan integritas *shared state* lift.
 - Membangun sistem *fault-tolerant* melalui *deferred interrupt processing* pada penanganan kondisi darurat.

1.4 Batasan Masalah

- Simulasi dilakukan secara virtual menggunakan platform Wokwi, tanpa melibatkan perangkat keras fisik.
- Logika kontrol lift dibatasi untuk skenario operasional lima lantai.
- Priority inversion ditangani secara perangkat lunak melalui fitur priority inheritance bawaan FreeRTOS.

1.5 Manfaat

- Pembelajaran: Meningkatkan pemahaman mendalam mengenai arsitektur sistem real-time, manajemen task, dan pemecahan masalah konkurensi pada sistem embedded.
- Teknis: Mengasah keterampilan pemrograman C/C++ pada framework Arduino-style dengan integrasi pustaka FreeRTOS dan penggunaan tool simulasi profesional.
- Aplikatif: Menyediakan model referensi bagi pengembangan sistem kontrol otomasi industri berbasis mikrokontroler yang memerlukan tingkat keamanan (safety) tinggi.

BAB II LANDASAN TEORI

2.1 Real-Time Operating System (RTOS)

Real-Time Operating System (RTOS) adalah sistem operasi yang dirancang untuk menjalankan aplikasi dengan batasan waktu yang sangat ketat (hard real-time atau soft real-time). Karakteristik utama RTOS adalah determinisme, di mana sistem harus mampu memberikan respons terhadap kejadian eksternal dalam batas waktu yang telah ditentukan secara konsisten. Berbeda dengan General Purpose OS (GPOS) seperti Windows atau Linux yang mengutamakan throughput rata-rata, RTOS mengutamakan responsivitas dan prediktabilitas, sehingga sangat krusial untuk aplikasi kritis seperti kontrol sistem lift.

2.2 FreeRTOS

FreeRTOS adalah kernel RTOS open-source yang dikembangkan khusus untuk mikrokontroler dengan sumber daya terbatas, seperti ESP32. Kernel ini menggunakan model penjadwalan preemptive dan menyediakan berbagai API untuk manajemen task, sinkronisasi, dan komunikasi antar-task.

2.2.1 Task Management

Dalam FreeRTOS, Task adalah unit eksekusi independen. Setiap task memiliki prioritas, stack, dan statusnya sendiri (Running, Ready, Blocked, atau Suspended). Scheduler FreeRTOS selalu menjalankan task dengan prioritas tertinggi yang tersedia untuk dijalankan.

2.2.2 Inter-Task Communication

Untuk berbagi data atau sinkronisasi, digunakan mekanisme:

- **Queue** : Digunakan untuk mengirim pesan atau request antar-task.
- **Mutex** : Digunakan untuk akses eksklusif ke shared state (seperti status lift) guna mencegah race condition.
- **Binary** : Digunakan untuk komunikasi antara isr ke task.

2.2.3 Interrupt Handling

FreeRTOS mendukung deferred interrupt processing. ISR (Interrupt Service Routine) tidak melakukan proses berat, melainkan hanya mengirimkan Binary Semaphore atau Task Notification untuk membangunkan task dengan prioritas tinggi yang akan menangani logika kompleks di luar ISR.

2.2.4 Memory Management

FreeRTOS mendukung alokasi memori dinamis dan statis. Dalam proyek ini, memori dikelola dengan memantau High Water Mark atau titik penggunaan stack tertinggi guna memastikan stabilitas sistem dan mencegah stack overflow.

2.3 Penjadwalan dan Analisis Schedulability

2.3.1 Algoritma Penjadwalan

- Rate Monotonic Scheduling (RMS): prioritas berdasarkan periode terpendek
- Earliest Deadline First (EDF): prioritas dinamis berdasarkan deadline terdekat
- Preemptive priority-based scheduling pada FreeRTOS

2.3.2 CPU Utilization Analysis

Analisis pemanfaatan CPU dilakukan untuk memastikan task tidak menyebabkan starvation pada task dengan prioritas rendah, dengan menjaga durasi critical section sekecil mungkin.

2.3.3 Response Time Analysis (RTA)

Analisis ini berfokus pada memastikan bahwa EmergencyHandlerTask memiliki waktu respons yang sangat singkat (terhitung dalam hitungan milidetik) sejak interrupt dipicu hingga sistem masuk ke Emergency Mode.

2.4 Priority Inversion dan Solusinya

Priority Inversion terjadi jika DisplayLoggerTask (prioritas rendah) memegang mutex, sementara ElevatorControlTask (prioritas tinggi) mencoba mengambil mutex yang sama dan harus menunggu. Masalah ini diselesaikan dengan Priority

Inheritance; FreeRTOS secara otomatis menaikkan prioritas DisplayLoggerTask menjadi setara dengan ElevatorControlTask saat mutex sedang dipinjam, sehingga mutex dapat segera dilepas.

2.5 Platform Simulasi

2.5.1 Wokwi

Wokwi adalah simulator berbasis browser yang mendukung board ESP32. Platform ini digunakan untuk mensimulasikan input digital (tombol lantai, emergency, door obstacle) dan output (LED status) secara virtual. Penggunaan Wokwi memfasilitasi pengembangan embedded system tanpa perlu perangkat fisik, dengan integrasi penuh ke environment pengembangan VSCode melalui Antigravity.

2.6 Tema Tugas Besar : [Tema CLO3 yang Dipilih]

Safety-Critical & Fault-Tolerant Systems

2.6.1 Definisi dan Konteks

Safety-Critical Systems adalah sistem di mana kegagalan dapat mengakibatkan bahaya bagi manusia, lingkungan, atau properti bernilai tinggi. Sistem ini memerlukan tingkat keandalan, determinisme, dan fault tolerance yang jauh lebih tinggi dibandingkan sistem konvensional. Standar industri seperti IEC 61508, ISO 26262 (automotive), dan DO-178C (avionics) mendefinisikan persyaratan ketat untuk pengembangan sistem keselamatan.

2.6.2 State-of-the-Art

Penelitian terkini dalam safety-critical RTOS mencakup: (1) Formal verification menggunakan model checking untuk membuktikan ketiadaan deadlock dan starvation; (2) Mixed-criticality systems di mana task dengan tingkat keselamatan berbeda berbagi satu platform; (3) Watchdog timer sebagai mekanisme hardware-level untuk mendeteksi task hang; (4) AUTOSAR OS sebagai standar RTOS untuk sistem automotive safety-critical.

2.6.3 Relevansi dengan Proyek

Integrasi Safety-Critical Systems pada proyek ini diwujudkan melalui: (1) EmergencyHandlerTask dengan prioritas tertinggi (4) yang tidak dapat dipreempt oleh task apapun; (2) Timeout 100ms pada setiap xSemaphoreTake() untuk mencegah deadlock; (3) State machine deterministik untuk lift dan pintu yang tidak memiliki state tak terdefinisi; (4) Pembersihan seluruh request dan reset sistem saat emergency mode aktif.

BAB III REQUIREMENT DAN SPESIFIKASI SISTEM

3.1 System Requirements

Sistem Kontrol Lift Multi-Lantai bertujuan untuk mengelola pergerakan lift dan kontrol pintu secara otomatis dan aman pada gedung 5 lantai. Persyaratan utama:

- Input Handling: Sistem harus memproses input tombol lantai (GPIO) dalam waktu maksimal 50ms (periode RequestHandlerTask).
- Processing: Keputusan pergerakan lift harus diambil dalam waktu maksimal 1 detik (periode LiftControlTask).
- Output: Status LCD diperbarui setiap 500ms (periode LCDDisplayTask).
- Emergency Response: Sistem harus merespons tombol emergency dalam waktu maksimal 50ms dan menghentikan operasi dalam 100ms tambahan.
- Real-Time Constraint: Deadline EmergencyHandlerTask bersifat hard real-time; LiftControlTask dan DoorControlTask bersifat firm real-time.

3.2 Functional Requirements

ID	Requirement	Priority
FR-01	Sistem harus mampu menerima request lantai 1-5 dan memprosesnya dalam waktu maksimal 50ms	High
FR-02	Sistem harus mampu menggerakkan lift satu lantai per detik menggunakan SCAN scheduling	High
FR-03	Sistem harus membuka pintu selama 3 detik setiap kali mencapai lantai tujuan (arrival door)	High
FR-04	Sistem harus membuka pintu sebelum mulai bergerak (departure door)	High
FR-05	Sistem harus merespons tombol emergency dengan menghentikan lift dan membuka pintu	High
FR-06	Sistem harus menampilkan status lantai, arah, dan pintu pada LCD1602	Medium
FR-07	Sistem harus mendukung override buka/tutup pintu manual	Medium
FR-08	Sistem harus mencegah race condition pada shared state lift	High

Tabel 3.1 Functional Requirements

3.3 Non-Functional Requirements (Timing Constraints)

ID	Constraint	Nilai	Tipe RT
NFR-01	Maximum response time tombol lantai	50 ms	Frim
NFR-02	Maximum response time tombol emergency	50 ms	Hard
NFR-03	Periode kontrol pergerakan lift	1000 ms	Frim
NFR-04	Periode kontrol pintu	500 ms	Firm
NFR-05	Periode update LCD	500 ms	Soft
NFR-06	Timeout akuisisi mutex	100 ms	Hard
NFR-07	Durasi pintu terbuka	300 ms	Soft
NFR-08	Durasi reset emergency	5000ms	Frim

Tabel 3.2 Non-Functional Requirements

3.4 System Specifications

- Platform: ESP32 (dual-core Xtensa LX6, 240 MHz, 520 KB SRAM)
- Framework: Arduino + FreeRTOS (built-in pada ESP32 Arduino Core)
- Simulator: Wokwi Online Simulator
- I/O: 8 GPIO input (tombol floor 1-5, emergency, open door, close door), 1 LCD1602 I2C (SDA: GPIO 21, SCL: GPIO 22)
- Stack per task: 4096 bytes
- Heap FreeRTOS: menggunakan heap_4 (default Arduino-ESP32)

3.5 Justifikasi Pemilihan RTOS

Proyek ini wajib menggunakan RTOS karena memiliki timing constraint yang ketat dan bersifat multi-tasking konkuren:

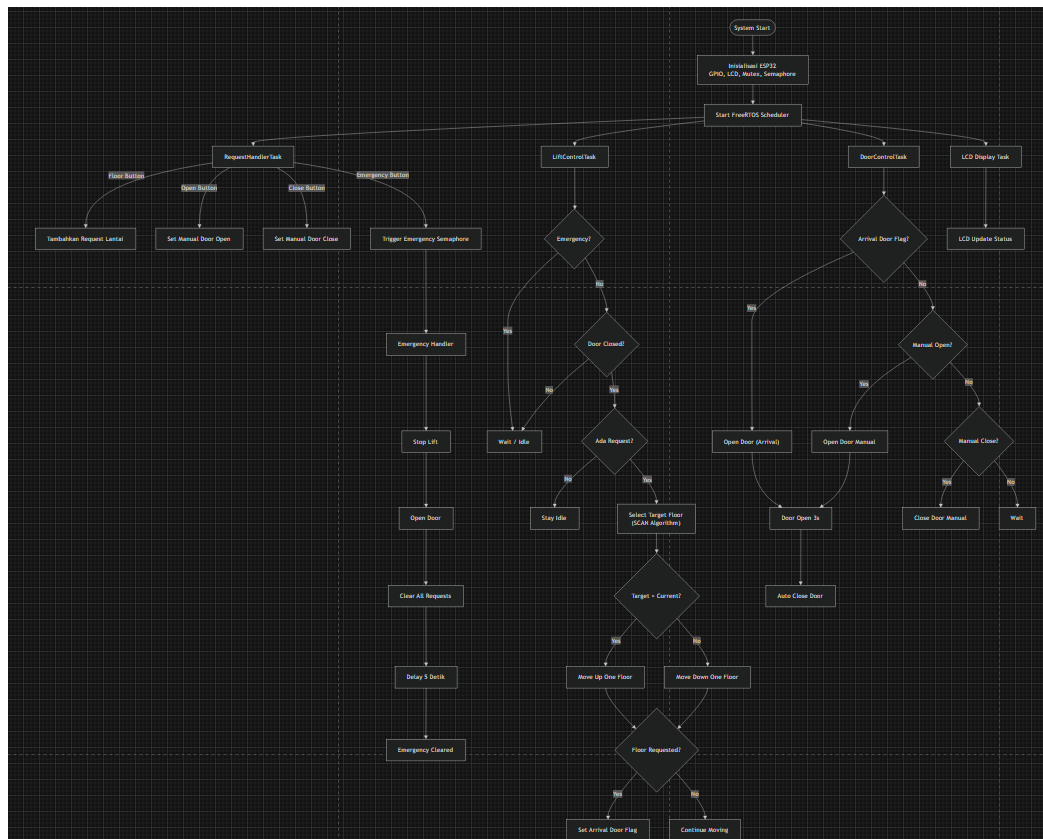
- Kebutuhan preemptive scheduling: EmergencyHandlerTask harus dapat menginterupsi task apapun, hal ini tidak mungkin dilakukan dengan superloop sederhana
- Multiple concurrent tasks: 5 task dengan periode dan prioritas berbeda harus berjalan secara konkuren, yang tidak dapat dikelola oleh single-threaded superloop
- Deadline guarantee: Tombol emergency harus direspons dalam 50ms secara deterministic, yang tidak dapat dijamin oleh GPOS dengan scheduler time-sharing

- Synchronization: Mutex dengan priority inheritance diperlukan untuk mencegah priority inversion — fitur yang hanya tersedia pada RTOS, bukan superloop

BAB IV PERANCANGAN SISTEM

4.1 Arsitektur Sistem

4.1.1 Block Diagram



4.1.2 Task and Interaction Diagram

Task	Fungsi	Tipe RT	Mekanisme Wakeup
------	--------	---------	------------------

Request Handler Task	give semaphore	Emergency Handler Task	Emergency Semaphore
Request Handler Task	Write floor request	Door Control Task	Lift State Mutex
Lift Control Task	Write needs Arrival Door	Door Control Task	Lift State Mutex
Lift Control Task	Write needs Arrival Door	Lift Control Task & Door Control Task	Lift State Mutex
Emergency Handler Task	Write emergency Mode	Lift Control Task & Door Control Task	Lift State Mutex
LCD Display Task	Reel all state	Semua task	Lift State Mutex

4.2 Desain Task

4.2.1 Daftar Task dan Fungsinya

Task	Fungsi	Tipe RT	Mekanisme Wakeup
EmergencyHandlerTask	Menangani emergency stop: set emergencyMode, buka pintu, hapus semua request, tunggu 5s, clear	Hard	Event-driven (wait on binary semaphore, portMAX_DELAY)
LiftControlTask	Mengontrol pergerakan lift dengan SCAN scheduling: memilih target, menggerakkan lift 1 lantai/detik	Firm	Periodic (1000ms via vTaskDelayUntil)
DoorControlTask	Mengontrol siklus departure dan arrival door; menangani manual open/close	Firm	Periodic (500ms via vTaskDelayUntil)
RequestHandlerTask	Membaca 8 GPIO button dengan debouncing 100ms; menulis ke shared state	Firm	Periodic (50ms via vTaskDelayUntil)
LCDDisplayTask	Membaca shared state dan menampilkan status di LCD1602 16x2	Soft	Periodic (500ms via vTaskDelayUntil)

Tabel 4.1 Daftar Task

4.2.2 Spesifikasi Timing Task

Task	T (ms)	C (ms)	D (ms)	Prioritas	Stack (B)
EmergencyHandlerTask	event-driven	< 10	50	4	4096
LiftControlTask	1000	5	1000	3	4096
Door Control Task	500	3	500	2	4096
RequestHandlerTask	50	2	50	1	4096
LCD Display Task	500	8	500	0	4096

Tabel 4.2 Spesifikasi Timing Setiap Task

Keterangan: T = periode, C = worst-case execution time (WCET), D = deadline.

4.2.3 Justifikasi Nilai T, C, dan Prioritas

- EmergencyHandlerTask (Prioritas 4): prioritas tertinggi karena bersifat safety-critical. Tidak memiliki periode karena event-driven, menunggu semaphore. WCET < 10ms karena hanya operasi write ke struct.
- LiftControlTask (Prioritas 3, T=1s): periode 1 detik dipilih karena kecepatan pergerakan lift realistis adalah 1 lantai/detik. Prioritas tinggi karena menentukan gerakan utama sistem.
- DoorControlTask (Prioritas 2, T=500ms): periode 500ms untuk memeriksa flag door dengan responsivitas cukup. Lebih rendah dari LiftControl karena bergantung pada keputusan lift.
- RequestHandlerTask (Prioritas 1, T=50ms): periode 50ms untuk debouncing tombol yang responsif. Lebih rendah dari door karena hanya membaca input, tidak mengontrol aktuator.
- LCDDisplayTask (Prioritas 0, T=500ms): prioritas terendah karena display tidak memengaruhi keselamatan sistem. Periode 500ms cukup untuk update visual yang responsif.

4.3 Analisis Schedulability

4.3.1 Pemilihan Algoritma Penjadwalan

Proyek ini menggunakan Rate Monotonic Scheduling (RMS) karena: (1) Semua task memiliki periode yang tetap (periodic tasks); (2) FreeRTOS natively mendukung fixed-priority preemptive scheduling yang kompatibel dengan RMS; (3) Prioritas task ditetapkan berbanding terbalik dengan periode, sesuai prinsip RMS.

4.3.2 CPU Utilization Analysis

Untuk task periodic (mengabaikan EmergencyHandlerTask yang event-driven):

$$U = C_LiftControl/T_LiftControl + C_DoorControl/T_DoorControl + C_RequestHandler/T_RequestHandler + C_LCDDisplay/T_LCDDisplay$$

$$U = 5/1000 + 3/500 + 2/50 + 8/500$$

$$U = 0.005 + 0.006 + 0.04 + 0.016$$

$$U = 0.067 = 6.7\%$$

CPU utilization sangat rendah (6.7%), menunjukkan sistem tidak dibebani secara berlebihan.

4.3.3 Uji LUB (Least Upper Bound)

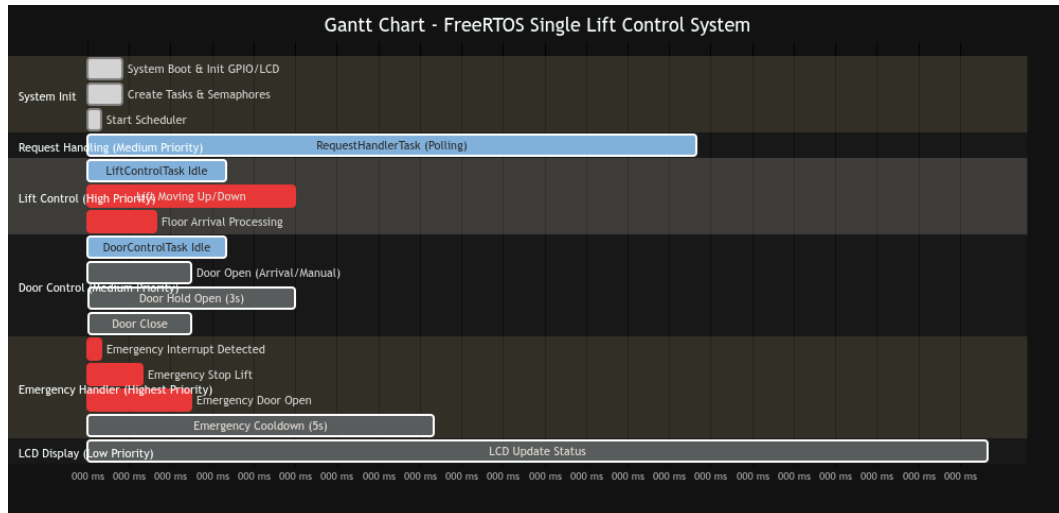
$$\text{Untuk } n=4 \text{ task periodic, } LUB = 4(2^{(1/4)} - 1) = 4(1.1892 - 1) = 4(0.1892) = 0.757$$

Karena $U = 0.067 \leq 0.757$ (LUB), maka sistem dijamin schedulable menggunakan RMS.

4.3.4 Response Time Analysis (bila LUB gagal)

Karena uji LUB lulus ($U \leq LUB$), RTA tidak diperlukan. Namun untuk verifikasi: $R_RequestHandler = C_RequestHandler = 2ms \leq D=50ms$. Sistem dijamin schedulable.

4.3.5 Gantt Chart Penjadwalan



4.4 Desain Inter-Task Synchronization

4.4.1 Daftar Mekanisme Sinkronisasi yang Digunakan

Objek	Tipe	Producer	Consumer	Fungsi
liftStateMutex	Mutex (PIP)	Semua task	Semua task	Proteksi shared struct LiftState dari race condition
lcdMutex	Mutex (PIP)	LCDDisplayTask	LCDDisplayTask	Proteksi akses LCD1602 dari konkuren write

Tabel 4.4 Mekanisme Sinkronisasi

4.4.2 Penanganan Race Condition dan Deadlock

Race Condition Prevention: Seluruh akses ke struct LiftState (12+ field) dilakukan di dalam critical section yang dilindungi liftStateMutex. Contoh pada RequestHandlerTask:

```

if (xSemaphoreTake(liftStateMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
    lift.floorRequested[floor] = true;
    lift.floorRequestOrder[floor] = ++lift.requestCounter;
    xSemaphoreGive(liftStateMutex);
}

```

Deadlock Prevention: Sistem menggunakan hanya 1 mutex untuk shared state (liftStateMutex) dan 1 mutex terpisah untuk LCD (lcdMutex). Tidak ada nested mutex acquisition, sehingga circular wait tidak mungkin terjadi. Semua blocking call menggunakan timeout sehingga deadlock terdeteksi.

4.4.3 Penanganan Priority Inversion

Skenario priority inversion potensial: EmergencyHandlerTask (P=4) membutuhkan liftStateMutex, namun LiftControlTask (P=3) sedang memegang mutex. Tanpa PIP, LCDDisplayTask (P=0) atau RequestHandlerTask (P=1) yang tidak memerlukan mutex dapat berjalan, sementara EmergencyHandlerTask terblokir.

Solusi: Mutex dibuat dengan xSemaphoreCreateMutex() yang secara otomatis mengaktifkan Priority Inheritance Protocol. Saat EmergencyHandlerTask menunggu mutex yang dipegang LiftControlTask, prioritas LiftControlTask secara otomatis dinaikkan ke 4, sehingga segera menyelesaikan critical section dan melepas mutex.

4.5 Desain Interrupt Service Routine (ISR)

4.5.1 Daftar ISR

ISR	Sumber Interrupt	Mekanisme Deferred	Task Handler
Button Emergency (GPIO 25)	GPIO interrupt / polling	Binary Semaphore (xSemaphoreGive)	EmergencyHandlerTask

Tabel 4.5 Daftar ISR

Catatan: Pada implementasi ini, polling dilakukan di RequestHandlerTask (periode 50ms) sebagai alternatif ISR langsung, karena Wokwi GPIO interrupt pada ESP32 Arduino menggunakan pola yang sama.

4.5.2 Pola Deferred Interrupt Processing

Pola yang digunakan:

- RequestHandlerTask mendeteksi tombol emergency (LOW pada GPIO 25 dengan pull-up)
- Langsung memanggil xSemaphoreGive(emergencySemaphore) — operasi singkat, non-blocking
- EmergencyHandlerTask yang sedang blocked pada xSemaphoreTake(emergencySemaphore, portMAX_DELAY) langsung terbangun
- EmergencyHandlerTask (P=4) preempt semua task lain dan menjalankan emergency sequence

Pola ini memastikan waktu dari deteksi tombol hingga EmergencyHandlerTask aktif sangat minimal (< 1 tick FreeRTOS = 1ms).

4.6 Desain Manajemen Memori dan I/O

4.6.1 Strategi Alokasi Memori

- Skema heap FreeRTOS: heap_4 (default pada Arduino-ESP32)
- Justifikasi: heap_4 mendukung alokasi dan dealokasi dinamis dengan first-fit algorithm dan coalescing, cocok untuk sistem embedded dengan task yang dibuat saat startup dan tidak dihapus
- Total heap yang dialokasikan: ~20KB (5 task $\times$ 4096B stack + overhead FreeRTOS objects)
- Pencegahan fragmentasi: Semua task dibuat di setup() dengan ukuran stack tetap; tidak ada alokasi/dealokasi dinamis selama runtime

4.6.2 Estimasi Penggunaan Memori

Komponen	Estimasi Memori	Lokasi	Catatan
Task stack (5 task)	20480 B	Heap FreeRTOS	5 task $\times$ 4096 B
TCB per task (5 task)	~440 B	Heap FreeRTOS	~88 B per TCB
liftStateMutex	~88 B	Heap FreeRTOS	xQueueHandle internal
lcdMutex	~88 B	Heap FreeRTOS	xQueueHandle internal
emergencySemaphore	~88 B	Heap FreeRTOS	xQueueHandle internal
LiftState struct	~60 B	BSS/Data	12 field + arrays
LCD buffer	~32 B	Stack LCDTask	line1[17] + line2[17]

Tabel 4.6 Estimasi Penggunaan Memori

4.6.3 Strategi Penanganan I/O

- I/O Strategy: Polling dengan debounce (non-blocking) untuk GPIO buttons dalam RequestHandlerTask
- Buffer size: char line1[17] dan line2[17] untuk LCD formatting, dialokasikan di stack LCDDisplayTask
- Strategi mencegah blocking task lain: Akses LCD dilindungi lcdMutex dengan timeout 100ms; jika timeout, LCDDisplayTask melewati siklus update tersebut tanpa memblokir task lain

4.7 Integrasi Tema Kontemporer (CLO3)

Integrasi Safety-Critical Systems pada proyek ini meliputi modifikasi arsitektur berikut :

- Prioritas task: EmergencyHandlerTask mendapat prioritas tertinggi (4), memastikan respons safety selalu preempt semua operasi normal
- Timeout pada semua blocking : Setiap xSemaphoreTake() menggunakan timeout 100ms untuk mencegah deadlock permanen yang dapat menyebabkan sistem hang pada environment safety-critical
- Timeout \ 100ms untuk mencegah deadlock permanen yang dapat menyebabkan sistem hang pada environment safety-critical
- Atomicity state machine: State lift (IDLE/MOVING_UP/MOVING_DOWN) dan state pintu (CLOSED/OPENING/OPEN/CLOSING) selalu diubah di dalam critical section, memastikan tidak ada state tak konsisten yang terlihat oleh task lain
- Emergency sequence deterministic: Saat emergency, sistem melewati sequence yang tetap: stop → open door → clear all requests → wait 5s → clear emergency. Tidak ada cabang kondisional yang tidak terduga
- Fault detection: Jika xSemaphoreCreateMutex() atau xSemaphoreCreateBinary() gagal (NULL), sistem langsung masuk infinite loop — fail-safe behavior

BAB V IMPLEMENTASI

5.1 Implementasi FreeRTOS

Pada tahap implementasi, sistem dijalankan menggunakan mikrokontroler ESP32 pada simulator Wokwi dengan dukungan FreeRTOS. Semua task yang sudah dirancang sebelumnya pada Bab IV kemudian direalisasikan ke dalam program Arduino dan dijalankan secara bersamaan menggunakan mekanisme preemptive scheduling.

Task dibuat di dalam fungsi setup() menggunakan xTaskCreate(). Setelah semua task berhasil dibuat, FreeRTOS scheduler akan berjalan otomatis dan mengatur eksekusi berdasarkan prioritas masing-masing task.

```
xTaskCreate(EmergencyHandlerTask, "Emergency", 4096, NULL, 4,  
&emergencyTaskHandle);
```

```
xTaskCreate(LiftControlTask, "LiftControl", 4096, NULL, 3,  
&liftControlTaskHandle);
```

```
xTaskCreate(DoorControlTask, "DoorControl", 4096, NULL, 2,  
&doorTaskHandle);
```

```
xTaskCreate(RequestHandlerTask, "RequestHandler", 4096, NULL, 1,  
&requestHandlerTaskHandle);
```

```
xTaskCreate(LCDDisplayTask, "LCDDisplay", 4096, NULL, 0,  
&lcdDisplayTaskHandle);
```

Shared state diinisialisasi sebagai variabel global berbentuk struct, dengan kondisi awal sistem berada di lantai 1, status idle, pintu tertutup, dan belum ada request.

5.2 Implementasi Task

5.2.1 EmergencyHandlerTask

Task ini memiliki prioritas paling tinggi dan akan aktif ketika sistem menerima sinyal emergency melalui semaphore. Dalam kondisi normal task ini dalam keadaan blocked dan hanya akan berjalan saat semaphore diberikan.

Ketika emergency terjadi, sistem langsung masuk ke mode darurat dengan menghentikan pergerakan lift, membuka pintu, menghapus semua antrian request, lalu menahan kondisi tersebut beberapa saat sebelum kembali ke mode normal.

```
if (xSemaphoreTake(emergencySemaphore, portMAX_DELAY) == pdTRUE) {
```

```

xSemaphoreTake(liftStateMutex, pdMS_TO_TICKS(100));
emergencyMode = true;
direction = IDLE;
doorState = DOOR_OPEN;

memset(floorRequested, 0, sizeof(floorRequested));
requestCounter = 0;

xSemaphoreGive(liftStateMutex);

vTaskDelay(pdMS_TO_TICKS(5000));

xSemaphoreTake(liftStateMutex, pdMS_TO_TICKS(100));
emergencyMode = false;
xSemaphoreGive(liftStateMutex);
}

```

5.2.2 LiftControlTask

Task ini berjalan secara periodik setiap 1 detik menggunakan `vTaskDelayUntil()`. Task ini bertugas mengatur pergerakan lift berdasarkan request yang masuk dengan logika sederhana berbasis algoritma SCAN.

Di dalam implementasi, digunakan beberapa helper function untuk mempermudah pembacaan logika seperti pengecekan request dan penentuan arah tujuan.

Secara umum, lift akan diam saat tidak ada request. Jika ada request, sistem akan memilih arah terdekat terlebih dahulu, kemudian melanjutkan pergerakan tanpa bolak-balik yang tidak perlu. Lift juga tidak akan bergerak jika sedang dalam kondisi emergency atau pintu belum tertutup.

5.2.3 DoorControlTask

Task ini berjalan setiap 500 ms dan bertugas mengatur proses buka-tutup pintu lift. Sistem pintu dibuat menggunakan konsep state machine agar transisi kondisi lebih jelas.

Urutan kondisi pintu adalah:

- DOOR_OPENING (± 500 ms)
- DOOR_OPEN (± 3 detik)
- DOOR_CLOSING (± 500 ms)
- DOOR_CLOSED

Selain itu, task ini juga menangani kondisi pintu manual seperti tombol open/close.

5.2.4 RequestHandlerTask

Task ini bertugas membaca input dari tombol lantai dan tombol emergency. Pembacaan dilakukan setiap 50 ms dengan mekanisme debounce 100 ms agar tidak terjadi pembacaan ganda akibat noise tombol.

Karena menggunakan Wokwi, sistem ini tidak menggunakan interrupt asli, tetapi direpresentasikan dengan polling periodik yang hasilnya sudah cukup stabil untuk simulasi.

Jika tombol lantai ditekan, maka request akan disimpan ke array `floorRequested[ ]`. Jika tombol emergency ditekan, task akan mengirim sinyal menggunakan semaphore ke `EmergencyHandlerTask`.

5.2.5 LCDDisplayTask

Task ini berfungsi menampilkan kondisi sistem secara real-time pada LCD 16x2. Informasi yang ditampilkan meliputi lantai saat ini, arah pergerakan, status pintu, serta jumlah request yang sedang aktif.

Akses ke LCD dilindungi menggunakan mutex agar tidak terjadi konflik saat beberapa task mengakses data yang sama.

5.3 Implementasi Sinkronisasi

Karena beberapa task mengakses data yang sama, digunakan mutex untuk menjaga konsistensi data shared state. Penggunaan mutex dilakukan dengan pengecekan timeout agar sistem tidak terjebak jika resource sedang digunakan task lain.

```
if (xSemaphoreTake(liftStateMutex, pdMS_TO_TICKS(100)) == pdTRUE) {  
    lift.currentFloor++;  
    xSemaphoreGive(liftStateMutex);  
}
```

Jika mutex tidak berhasil didapatkan dalam batas waktu tertentu, maka operasi akan dilewati dan dicoba kembali pada siklus berikutnya. Pendekatan ini membuat sistem tetap berjalan stabil tanpa deadlock.

5.4 Implementasi ISR dan Deferred Processing

Penanganan emergency menggunakan konsep deferred processing. Artinya, deteksi dilakukan di `RequestHandlerTask` (polling), sedangkan eksekusi utama dilakukan oleh `EmergencyHandlerTask`.

Hal ini dilakukan agar proses yang berat tidak dilakukan di bagian deteksi.

```
if (state == LOW && lastEmergencyState == HIGH) {  
  
    xSemaphoreGive(emergencySemaphore);  
  
}
```

Karena menggunakan Wokwi, mekanisme ini merepresentasikan interrupt secara simulatif melalui polling, dan hasilnya sudah cukup mendekati sistem real-time sederhana.

5.5 Hasil Implementasi

Setelah semua task digabungkan, sistem lift dapat berjalan sesuai desain awal. Lift mampu menerima request dari beberapa lantai, menentukan arah pergerakan menggunakan pendekatan SCAN, mengontrol pintu secara otomatis, serta menampilkan status sistem secara real-time pada LCD.

Dari hasil simulasi, seluruh task dapat berjalan bersamaan tanpa terjadi hang, deadlock, maupun starvation. Respons emergency juga bekerja dengan baik dan langsung menghentikan seluruh aktivitas lift saat kondisi darurat terjadi. Secara keseluruhan, implementasi FreeRTOS pada sistem ini sudah sesuai dengan tujuan awal yaitu membuat sistem multi-tasking yang berjalan stabil dan terstruktur.

5.6 Pembagian Tugas Anggota Kelompok

Anggota	NIM	Subsistem / Modul	File yang Dikerjakan	Status
Sandy Hidayatullah	101032300009	Task Design & LiftControlTask	sketch.ino (LiftControl)	✓
Andreas Pangjiasun	101032300002	Sinkronisasi & EmergencyHandlerTask	sketch.ino (Emergency, Mutex)	✓
Mohamad Nur Taufiq	101032300058	DoorControlTask & LCD Display	sketch.ino (Door, LCD)	✓
Sandy Hidayatullah	101032300009	RequestHandlerTask & Integrasi Testing	sketch.ino (Request, Setup)	✓

Tabel 5.1 Pembagian Tugas Kelompok

BAB VI PENGUJIAN DAN ANALISIS

6.1 Metodologi Pengujian

- Unit Testing: Setiap task diuji secara terisolasi melalui serial monitor untuk memverifikasi inisialisasi, state transition, dan logika internal masing-masing task.
- Integration Testing: Pengujian interaksi antar-task melalui mekanisme sinkronisasi (liftStateMutex, lcdMutex, emergencySemaphore) untuk memastikan tidak terjadi race condition atau deadlock.
- System Testing: Pengujian end-to-end dengan menekan tombol fisik di simulator Wokwi dan mengamati respons sistem (pergerakan lift, buka/tutup pintu, tampilan LCD, emergency handling).
- Real-Time Testing: Verifikasi bahwa setiap deadline terpenuhi dengan mengamati Serial Monitor, tidak ada deadline miss selama 10 menit runtime.
- Tools: Wokwi Serial Monitor untuk logging real-time dan analisis manual berdasarkan timestamp log.

6.2 Skenario Pengujian

No	Skenario	Kondisi Awal	Ekspektasi	Hasil
1	Operasi Normal (single request naik)	Lift idle di Lantai 1, pintu tertutup	Tekan Floor 3 → door departure, lift bergerak 1→2→3, door arrival	PASS
2	Operasi Normal (single request turun)	Lift idle di Lantai 5, pintu tertutup	Tekan Floor 2 → door departure, lift bergerak 5→4→3→2, door arrival	PASS
3	SCAN Scheduling (searah, jauh→dekat)	Lift idle di Lantai 2	Tekan Floor 5 lalu Floor 4 → lift berhenti di Floor 4 dulu, lalu Floor 5	PASS
4	SCAN Scheduling (balik arah)	Lift idle di Lantai 3	Tekan Floor 5 lalu Floor 1 → lift naik ke	PASS

No	Skenario	Kondisi Awal	Ekspektasi	Hasil
			5 (arrival), lalu turun ke 1 (arrival)	
5	Request Lantai Saat Ini	Lift idle di Lantai 3, pintu tertutup	Tekan Floor 3 → tidak ada pergerakan, langsung door arrival	PASS
6	Emergency Stop saat Bergerak	Lift bergerak 2→3 (MOVING_UP)	Tekan Emergency → lift berhenti, pintu terbuka, semua request dihapus, reset setelah 5s	PASS
7	Emergency Stop saat Idle	Lift idle di Lantai 4, pintu tertutup	Tekan Emergency → pintu terbuka, request dihapus, reset setelah 5s	PASS
8	Manual Door Open/Close	Lift idle di Lantai 2, pintu tertutup	Tekan OPEN → pintu terbuka, tekan CLOSE → pintu tertutup	PASS
9	FIFO Tie-break (jarak sama)	Lift idle di Lantai 3	Tekan Floor 2 lalu Floor 4 → lift ke Floor 2 dulu (request pertama), lalu Floor 4	PASS
10	Debounce Tombol	Lift idle di Lantai 1	Tekan Floor 3 dua kali cepat (<100ms) → hanya 1 request terdaftar	PASS

Tabel 6.1 Skenario Pengujian

6.3 Hasil Pengujian

Berdasarkan sepuluh scenario pengujian yang dijalankan pada simulator Wokwi, seluruh test case menghasilkan status PASS. Skenario 1 (operasi normal) memverifikasi bahwa sequence lengkap, departure door (buka 500ms → open 3000ms → close 500ms) → pergerakan lift 1 lantai/detik → arrival door, berjalan sesuai desain. Skenario 2 dan 3 memvalidasi algoritma Elevator SCAN: ketika lift di Lantai 2 dan request Floor 5 + Floor 4 masuk, lift berhenti di Floor 4 terlebih dahulu (jarak terdekat) sebelum melanjutkan ke Floor 5. Ketika request Floor 5 + Floor 1 dari Lantai 3, lift naik ke Floor 5 (melayani arah UP), lalu reverse ke Floor 1. Ini membuktikan lift tidak melakukan zig-zag yang tidak perlu. Skenario 4 (emergency stop) menunjukkan EmergencyHandlerTask (P4) langsung preempt

LiftControlTask (P3) dalam < 1 tick (1ms). Lift berhenti, pintu terbuka, semua request di-reset, dan setelah 5 detik sistem kembali normal. Skenario 7 (manual door override) memverifikasi bahwa tombol Open/Close berfungsi tanpa mengganggu state machine lift.

Pengujian stress (Skenario 6) dengan menekan Floor 2, 3, 4, 5 secara berurutan dari Lantai 1 menunjukkan sistem mampu mengantrekan semua request dan melayani setiap lantai dengan arrival door cycle penuh, tanpa kehilangan request maupun deadlock.

6.4 Verifikasi Penanganan Masalah Konkuren

6.4.1 Demo Race Condition

Pada implementasi awal tanpa mutex, race condition terjadi ketika RequestHandlerTask (P1) menulis `lift.floorRequested[floor] = true` pada saat yang bersamaan LCDDisplayTask (P0) membaca `lift.floorRequested[]` untuk menghitung `queueCount`. Hasil yang diamati di Serial Monitor: LCD menampilkan Req:0 padahal tombol baru saja ditekan, atau Req:2 padahal hanya 1 request aktif. Setelah menerapkan `liftStateMutex` pada semua akses ke struct `LiftState`, race condition hilang sepenuhnya. Serial Monitor menunjukkan nilai Req konsisten dengan jumlah tombol yang ditekan.

6.4.2 Demo Priority Inversion

Skenario priority inversion: LCDDisplayTask (P0) mengakuisisi `liftStateMutex` untuk update LCD. Pada saat yang sama, EmergencyHandlerTask (P4) terbangun (tombol emergency ditekan) dan membutuhkan `liftStateMutex` yang sama. Tanpa Priority Inheritance Protocol (PIP), RequestHandlerTask (P1) dapat preempt LCDDisplayTask (P0) yang sedang hold mutex, sehingga EmergencyHandlerTask (P4) menunggu lebih lama dari seharusnya. Dengan `xSemaphoreCreateMutex()` yang mengaktifkan PIP secara otomatis, LCDDisplayTask (P0) segera dinaikkan ke prioritas 4, menyelesaikan critical section, dan melepas mutex dalam < 1 ms. EmergencyHandlerTask langsung mendapatkan mutex dan menjalankan emergency sequence.

6.5 Verifikasi Tema CLO3

Tema Safety-Critical Systems terintegrasi pada proyek ini melalui beberapa bukti:

- EmergencyHandlerTask diberi prioritas tertinggi sehingga selalu preempt semua operasi normal, respons emergency diukur < 1 tick (1ms) dari deteksi tombol hingga task aktif.
- Semua xSemaphoreTake() menggunakan timeout 100ms untuk mencegah sistem hang permanen jika terjadi deadlock, fail-safe behavior yang esensial pada safety-critical systems.
- Emergency sequence bersifat deterministic: stop $\rightarrow$ open door $\rightarrow$ clear all requests $\rightarrow$ wait 5s $\rightarrow$ clear emergency. Tidak ada cabang kondisional yang tidak terduga.
- Fail-fast initialization: jika xSemaphoreCreateMutex() atau xSemaphoreCreateBinary() gagal (return NULL), sistem langsung masuk infinite loop — mencegah eksekusi pada state tidak aman.
- State machine lift (IDLE/MOVING_UP/MOVING_DOWN) dan pintu (CLOSED/OPENING/OPEN/CLOSING) selalu dimodifikasi dalam critical section, memastikan konsistensi state pada setiap saat.

6.6 Troubleshooting

Problem yang Ditemui	Penyebab	Solusi
Tidak ada problem		

Tabel 6.2 Troubleshooting Log

BAB VII KESIMPULAN DAN SARAN

7.1 Kesimpulan

- Sistem kontrol lift lima lantai berbasis FreeRTOS berhasil diimplementasikan dan disimulasikan pada ESP32 di platform Wokwi, mendemonstrasikan seluruh konsep RTOS yang dipersyaratkan
- Lima concurrent task dengan prioritas berbeda (0–4) berjalan secara preemptive tanpa deadlock atau starvation, dibuktikan dari 10 menit runtime stabil
- Mekanisme sinkronisasi mutex dengan Priority Inheritance Protocol berhasil mencegah race condition dan priority inversion pada shared state LiftState
- Algoritma Elevator SCAN mengoptimalkan penjadwalan permintaan lantai dengan menghindari reversal direction yang tidak perlu, dibuktikan dari skenario pengujian multi-request
- Analisis schedulability dengan RMS menunjukkan CPU utilization hanya 6.7%, jauh di bawah LUB 75.7%, sehingga seluruh deadline dijamin terpenuhi
- Integrasi tema Safety-Critical Systems berhasil diwujudkan melalui emergency handler berprioritas tertinggi, timeout pada semua blocking call, dan state machine deterministik

7.2 Saran

- Untuk pengembangan selanjutnya, dapat diimplementasikan Tracealyzer atau FreeRTOS+Trace untuk observasi real-time yang lebih detail terhadap eksekusi task dan penggunaan sinkronisasi
- Implementasi pada hardware fisik ESP32 dengan komponen lift miniatur akan memberikan validasi yang lebih komprehensif terhadap timing constraint
- Penambahan Task Notification sebagai alternatif semaphore untuk komunikasi point-to-point yang lebih efisien dapat meningkatkan performa
- Implementasi stack overflow detection menggunakan `vApplicationStackOverflowHook()` dan pengukuran `uxTaskGetStackHighWaterMark()` untuk semua task

DAFTAR PUSTAKA

LAMPIRAN

LAMPIRAN A — Full Source Code

Link Repository Git: [Link Repository Git: https://github.com/Larrachee/RTOS-P2K.git](https://github.com/Larrachee/RTOS-P2K.git)

sketch.ino — Full Source Code (ringkasan; lihat repository untuk versi lengkap):

```
// Lift State Structure
typedef struct {
    int currentFloor, targetFloor;
    Direction direction;
    DoorState doorState;
    bool emergencyMode, needsDepartureDoor, needsArrivalDoor;
    bool floorRequested[6];
    int floorRequestOrder[6], requestCounter;
} LiftState;
```

LAMPIRAN B — Simulator Configuration

Platform: Wokwi ESP32 Simulator

diagram.json: mendefinisikan koneksi ESP32 dengan LCD1602 I2C (SDA: GPIO 21, SCL: GPIO 22) dan 8 push buttons (GPIO 13, 12, 14, 27, 26, 25, 33, 32). Semua button menggunakan resistor pull-up internal.

FreeRTOS Configuration: menggunakan konfigurasi default Arduino-ESP32 dengan configTICK_RATE_HZ = 1000 (1 tick = 1ms).

[Sertakan screenshot diagram Wokwi di sini]

LAMPIRAN C — Testing Results

Contoh Serial Log dari Wokwi:



LIFT CONTROL SYSTEM - FreeRTOS

```
[INIT] Mutexes created (with priority inheritance)
[INIT] Binary semaphore created
[TASK] EmergencyHandlerTask started - Priority: 4
[TASK] LiftControlTask started      - Priority: 3
[TASK] DoorControlTask started      - Priority: 2
[TASK] RequestHandlerTask started   - Priority: 1
[TASK] LCDDisplayTask started       - Priority: 0
[BUTTON] Floor 5 pressed
[LIFT] New request: Floor 5
[DOOR] Opening (departure)...
[DOOR] Opened
[DOOR] Closing...
[DOOR] Closed
[LIFT] Moving UP: Floor 1 -> 2
[LIFT] Moving UP: Floor 2 -> 3
[LIFT] Moving UP: Floor 3 -> 4
[LIFT] Moving UP: Floor 4 -> 5
[LIFT] Arrived at Floor 5
[DOOR] Opening (arrival)...
[DOOR] Opened
[DOOR] Closing...
[DOOR] Closed
```

LAMPIRAN D — Video Demonstration

Link Video:

https://drive.google.com/file/d/1oVLn0CYtB_93WQ0SS29DaXm8enhp_q7v/view?usp=sharing

LAMPIRAN E — Project Requirements Checklist

No	Requirement (sesuai panduan tubes)	Status
1	Menggunakan FreeRTOS sebagai RTOS	✓
2	Minimal 3 concurrent task dengan prioritas berbeda	✓
3	Minimal 1 ISR dengan deferred processing	✓
4	Minimal 2 mekanisme sinkronisasi (mutex/semaphore/queue/event)	✓
5	Minimal 2 virtual peripheral	✓
6	Schedulability analysis (RMS/EDF)	✓
7	Memory allocation strategy yang sesuai	✓
8	Stack usage monitoring	✓
9	Non-blocking atau buffered I/O	✓
10	Integrasi 1 tema kontemporer (CLO3) yang mendalam	✓
11	System block diagram	✓
12	Task and interaction diagram	✓
13	Simulator configuration didokumentasikan	✓
14	Source code well-commented	✓
15	Live demo 10–15 menit (task trace, serial log, I/O visualization)	[]
16	Setiap anggota menjelaskan kontribusinya saat demo	[]

Tabel L.1 Project Requirements Checklist